

2.2 Unconstrained Methods

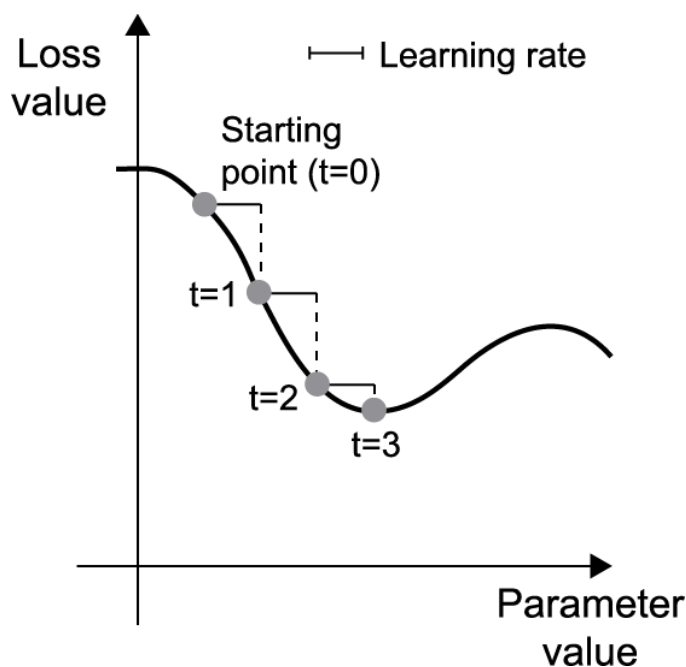
Line search, steepest descent, momentum, and SGD.

These notes walk through the lecture slides. The original deck is unchanged; use **(slides)** on the course hub if you want that PDF.

The figures follow Deisenroth, Faisal, and Ong, *Mathematics for Machine Learning*.

1. Line search

Draw x , predict y_{pred} , score the mismatch with J , take the gradient, and step **against** it.

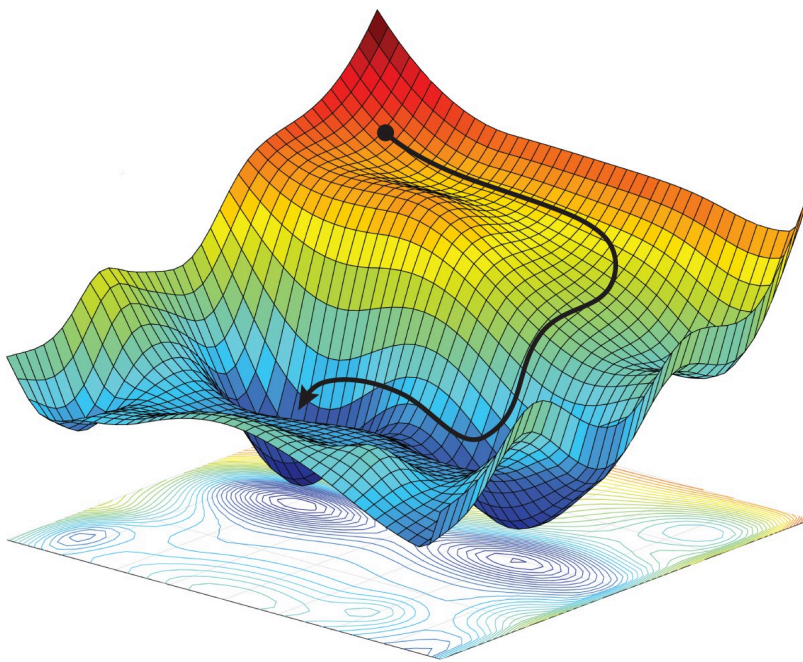


Almost every method in this note is

$$\theta^{(k+1)} = \theta^{(k)} + \alpha^{(k)} \mathbf{s}^{(k)}.$$

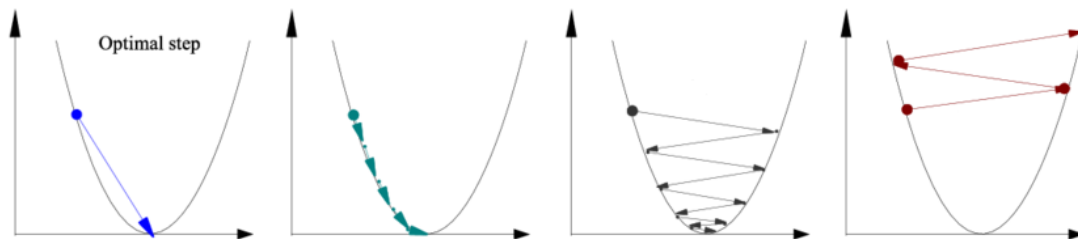
1. Pick a **direction** $\mathbf{s}^{(k)}$.
2. Pick a **step** $\alpha^{(k)}$ (learning rate) that lowers $J(\theta^{(k)} + \alpha \mathbf{s}^{(k)})$ in α .
3. Take the step.

A direction is a **descent** direction when $\mathbf{s}^\top \nabla J < 0$. **Steepest descent** (gradient descent) uses $\mathbf{s} = -\nabla J$.



2. Why steepest descent zigzags

Steepest descent can be slow. The step length matters, and the **asymptotic** rate is worse than Newton-type methods. On a long thin valley the problem is **poorly conditioned**: $-\nabla J$ points into the walls, almost orthogonal to the short path to the min. Exact line search still hops.



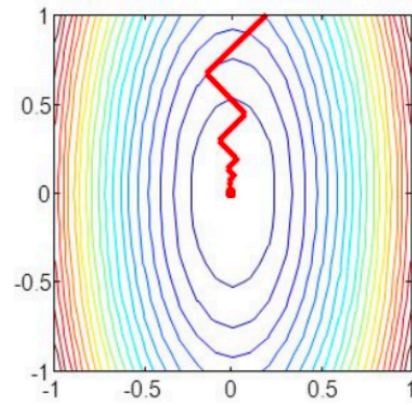
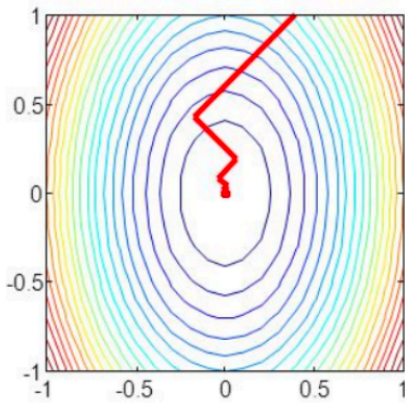
See it on a quadratic. Let

$$J(\theta) = \frac{1}{2}\theta^T \mathbf{Q}\theta - c$$

with $\mathbf{Q}$ symmetric positive definite. Diagonalize $\mathbf{Q} = \mathbf{U}\mathbf{\Lambda}\mathbf{U}^T$. In coordinates $\mathbf{z} = \mathbf{U}^T\theta$, minimizing J is the same as minimizing $\sum_i \lambda_i z_i^2$. In two dimensions the contours are ellipses

$$\lambda_1 z_1^2 + \lambda_2 z_2^2 = c.$$

The axis ratio is the **condition number** of $\mathbf{Q}$ (largest over smallest eigenvalue). Large condition number: a long thin valley, tiny progress.



3. Momentum

Remember the last step and refuse to turn on a dime (Rumelhart et al., 1986):

$$\theta^{(k+1)} = \theta^{(k)} - \alpha^{(k)} \nabla J(\theta^{(k)}) + \beta \Delta \theta^{(k)},$$

$$\Delta \theta^{(k)} = \theta^{(k)} - \theta^{(k-1)} = \beta \Delta \theta^{(k-1)} - \alpha^{(k-1)} \nabla J(\theta^{(k-1)}),$$

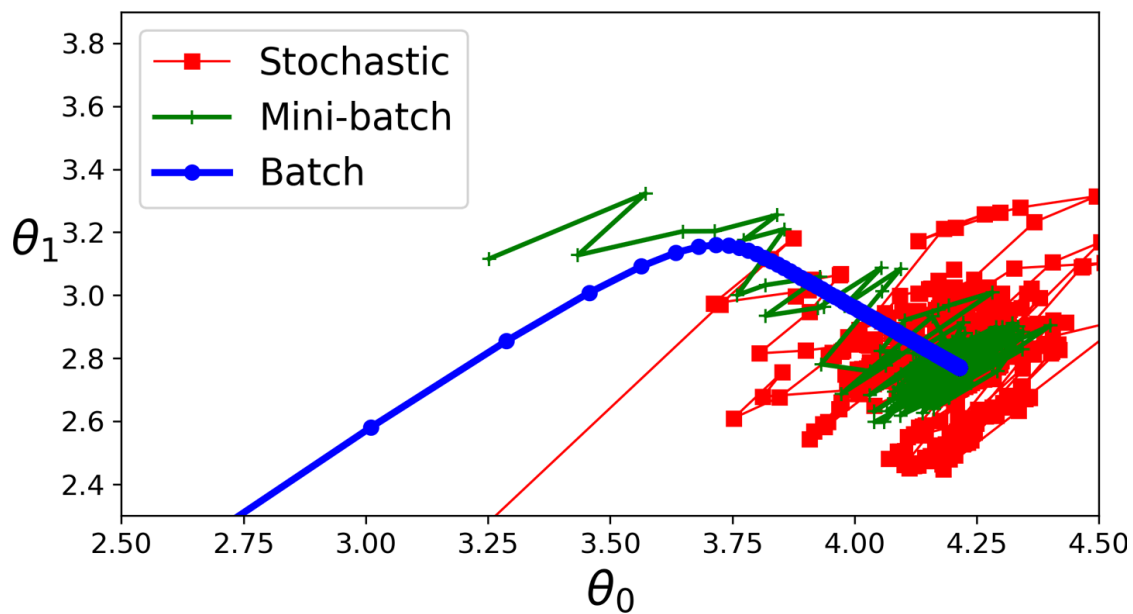
with $\beta \in [0, 1]$. A **heavy ball** in a valley: oscillations damp, progress along the floor improves. Noisy gradients benefit because the memory averages them. Stochastic approximations of ∇J are the usual case, not a special case.

4. SGD and mini-batches

If $J = \sum_{n=1}^N J_n$, **batch** gradient descent uses the full sum. That is expensive when N is large and there is no closed form for ∇J .

SGD uses one term (or a random subset) as a noisy stand-in for ∇J . With a decaying step size, SGD converges almost surely to a local min under mild assumptions.

Mini-batch gradient descent sits in between: a random subset, not one row and not all N . Still an unbiased estimate of the true gradient. Cheaper than batch, less noisy than one-row SGD.



Why accept an approximate gradient?

Mini-batch	What you buy	What you pay
Large	Accurate ∇J , stable steps, fast matrix kernels	Each step is expensive; easy to sit in a sharp hole
Small	Cheap steps; noise can leave a bad local min; often enough for generalization	Higher variance in the updates

Memory and wall-clock are the practical reasons. Mini-batch size is a sample size for an empirical mean.

5. Practice

1. What does a large condition number do to steepest descent?
2. Write the descent test $\mathbf{s}^\top \nabla J < 0$ in words. Why does $\mathbf{s} = -\nabla J$ pass it whenever you are not already at a critical point?
3. Name one reason to prefer a small mini-batch over a large one, and one reason for the opposite.